

Relatório da Sprint 1

Modelo Entidade-Relacionamento do Sistema CAMAAR

Arthur Souza Chagas – Matrícula: 221037385

João Felipe Stein – Matrícula: 241039331

Luidgi Varela Carneiro – Matrícula: 231011669

Yuri Santana Lopes – Matrícula: 222009750

João Vitor das Neves Romero – Matrícula: 221028546

Maio de 2026

Introdução

O **CAMAAR** é um sistema de gestão acadêmica desenvolvido no contexto da disciplina de Engenharia de Software. Seu objetivo é informatizar e centralizar processos ligados à aplicação de formulários de avaliação acadêmica, ao gerenciamento de usuários (administradores, professores e estudantes), à organização de departamentos, matérias e turmas, e ao controle de respostas submetidas por discentes e docentes.

As funcionalidades do sistema foram organizadas em dois épicos principais. O **Épico Usuário** abrange o cadastro de usuários no sistema, o sistema de *login*, a definição e a redefinição de senha. O **Épico Formulário** abrange a criação de *templates* de formulário, a criação de formulários de avaliação, a resposta de formulários pelos usuários, a visualização de resultados e a geração de relatórios pelo administrador.

Este relatório apresenta o modelo entidade-relacionamento (ER) elaborado para o sistema, descrevendo suas principais entidades, atributos e relacionamentos com base nas histórias de usuário definidas para a primeira sprint. Adicionalmente, é apresentada uma investigação técnica (*spike*) sobre a implementação das *views* em Ruby on Rails a partir do protótipo disponível no Figma, como item de ponto extra.

Explicação do Sistema

O CAMAAR é uma aplicação *web* desenvolvida em *Ruby on Rails*. O sistema permite que administradores cadastrem e gerenciem usuários, departamentos, matérias e turmas, além de criarem *templates* de formulários de avaliação. Professores, por sua vez, podem utilizar esses *templates* para criar formulários específicos destinados às suas turmas. Estudantes matriculados nas turmas podem acessar e responder os formulários disponibilizados, e os resultados podem ser visualizados tanto pelos criadores quanto pelos próprios respondentes, conforme a configuração do formulário.

A divisão de responsabilidades do sistema segue uma hierarquia de papéis: **Administrador**, **Professor** e **Estudante**, todos derivados de uma entidade base **Usuário**. Administradores e Professores herdam de **CriadorDeFormulário**, indicando que ambos têm permissão para criar formulários.

Modelo Entidade-Relacionamento

A seguir são descritas as principais entidades do sistema CAMAAR identificadas a partir das histórias de usuário da sprint.

1. Usuário

- **id**: Identificador único (chave primária, Long).
- **email**: Endereço de e-mail do usuário (String).
- **matricula**: Código de matrícula institucional (String).
- **senha**: Senha armazenada com hash de 32 caracteres (hash32).
- **nome**: Nome completo do usuário (String).
- **role**: Papel do usuário no sistema — Admin, Professor ou Estudante (Enum).

Relacionamento: entidade base do sistema. **Admin**, **Professor** e **Estudante** herdam de Usuário. Admin e Professor herdam adicionalmente de **CriadorDeFormulário**.

2. CriadorDeFormulário

Entidade intermediária que representa usuários com permissão para criar formulários. É herdada por **Admin** e **Professor**.

Relacionamento: associada a **Template de Formulário** e **Formulário** por meio do atributo `idCriadorDeFormulario`.

3. Admin

- **idDepartamento**: Chave estrangeira referenciando o departamento ao qual o administrador está vinculado (Long FK).

Relacionamento: herda de **CriadorDeFormulário** e está vinculado a um **Departamento**.

4. Professor

Herda todos os atributos de **CriadorDeFormulário** e, por extensão, de **Usuário**. Pode ser responsável por uma ou mais **Turmas** e pode criar **Formulários**.

5. Estudante

- **matricula**: Identificador próprio do estudante (String).

Relacionamento: herda de **Usuário** e se associa a **Turmas** por meio da entidade intermediária **Matrícula**. Pode submeter respostas a formulários via **Resposta Formulário**.

6. Departamento

- **id**: Identificador único (chave primária, Long).
- **nomeDepartamento**: Nome do departamento acadêmico (String).

Relacionamento: um departamento pode ter várias **Matérias** e está associado a um ou mais **Admins**.

7. Matéria

- **id**: Identificador único (chave primária, Long).
- **codigoMateria**: Código identificador da matéria (String).
- **nomeMateria**: Nome da disciplina (String).
- **descricaoMateria**: Descrição da ementa ou conteúdo (text).
- **idDepartamento**: Chave estrangeira referenciando o departamento responsável (Long FK).

Relacionamento: pertence a um **Departamento** e pode ter várias **Turmas** associadas.

8. Turma

- **id**: Identificador único (chave primária, Long).
- **numeroTurma**: Número identificador da turma (Int).
- **semestre**: Semestre letivo da turma (String).
- **notaTotal**: Nota total da turma (Double).
- **idMateria**: Chave estrangeira referenciando a matéria (Long FK).
- **idProfessor**: Chave estrangeira referenciando o professor responsável (Long FK).

Relacionamento: pertence a uma **Matéria**, é ministrada por um **Professor** e possui **Estudantes** vinculados via **Matrícula**. Pode ter **Formulários** associados.

9. Matrícula

Entidade intermediária que representa o vínculo de um estudante a uma turma, implementando o relacionamento *many-to-many* entre **Estudante** e **Turma**.

- **idEstudante**: Chave estrangeira referenciando o estudante (Long PK).
- **idTurma**: Chave estrangeira referenciando a turma (Long PK).
- **trancado**: Indica se a matrícula está trancada (boolean).

10. Template de Formulário

- **id**: Identificador único (chave primária, Long).
- **nomeTemplate**: Nome do *template* (String).
- **listaFormulariosCriados**: Lista de formulários gerados a partir deste *template* (List<FK>).
- **idCriadorDeFormulario**: Chave estrangeira referenciando o criador (Long FK).

Relacionamento: criado por um **CriadorDeFormulário** e contém **Perguntas**. Serve de base para a criação de **Formulários**.

11. Pergunta

- **id**: Identificador único (chave primária, Long).
- **Enunciado**: Texto da pergunta (String).
- **tipoPergunta**: Tipo da pergunta — discursiva ou de múltipla escolha (Enum).
- **gabaritoDiscursiva**: Gabarito para perguntas discursivas (String).
- **gabaritoRadio**: Gabarito para perguntas de múltipla escolha (Int).
- **opcoesRadio**: Lista de opções de resposta (List<String>).
- **idTemplateFormulario**: Chave estrangeira referenciando o *template* ao qual pertence (Long FK).

Relacionamento: pertence a um **Template de Formulário**.

12. Formulário

- **id**: Identificador único (chave primária, Long).
- **nomeFormulario**: Nome do formulário (String).
- **notaTotal**: Nota máxima do formulário (Int).
- **notaObtida**: Nota obtida pelo respondente (Int).
- **publicoEstudante**: Indica se o formulário é visível a estudantes (boolean).
- **idTemplateFormulario**: Chave estrangeira referenciando o *template* de origem (Long FK).
- **idTurma**: Chave estrangeira referenciando a turma destinatária (Long FK).
- **idCriadorDeFormulario**: Chave estrangeira referenciando o criador (Long FK).

Relacionamento: gerado a partir de um **Template de Formulário**, associado a uma **Turma** e a um **CriadorDeFormulário**. Contém **PerguntasFormulário** e recebe **Respostas Formulário**.

13. PerguntaFormulário

- **id**: Identificador único (chave primária, Long).
- **Enunciado**: Texto da pergunta no formulário (String).
- **tipoPergunta**: Tipo da pergunta (Enum).
- **gabaritoDiscursiva**: Gabarito discursivo (String).
- **gabaritoRadio**: Gabarito de múltipla escolha (Int).
- **opcoesRadio**: Opções de resposta (List<String>).
- **idFormulario**: Chave estrangeira referenciando o formulário (Long FK).

Relacionamento: pertence a um **Formulário** e está associada a **Respostas Pergunta**.

14. Resposta Formulário

- **id**: Identificador único (chave primária, Long).
- **dataResposta**: Data e hora de submissão (DateTime).
- **idFormulario**: Chave estrangeira referenciando o formulário respondido (Long FK).
- **idUsuario**: Chave estrangeira referenciando o usuário que respondeu (Long FK).

Relacionamento: associada a um **Formulário** e a um **Usuário**. Contém uma ou mais **Respostas Pergunta**.

15. Resposta Pergunta

- **id**: Identificador único (chave primária, Long).
- **tipoResposta**: Tipo da resposta — discursiva ou de múltipla escolha (Enum).
- **respostaDiscursiva**: Conteúdo textual da resposta discursiva (String).
- **respostaRadio**: Índice da opção selecionada em múltipla escolha (Int).
- **idRespostaFormulario**: Chave estrangeira referenciando a resposta ao formulário (Long FK).
- **idPerguntaFormulario**: Chave estrangeira referenciando a pergunta respondida (Long FK).

Relacionamento: pertence a uma **Resposta Formulário** e está associada a uma **PerguntaFormulário**.

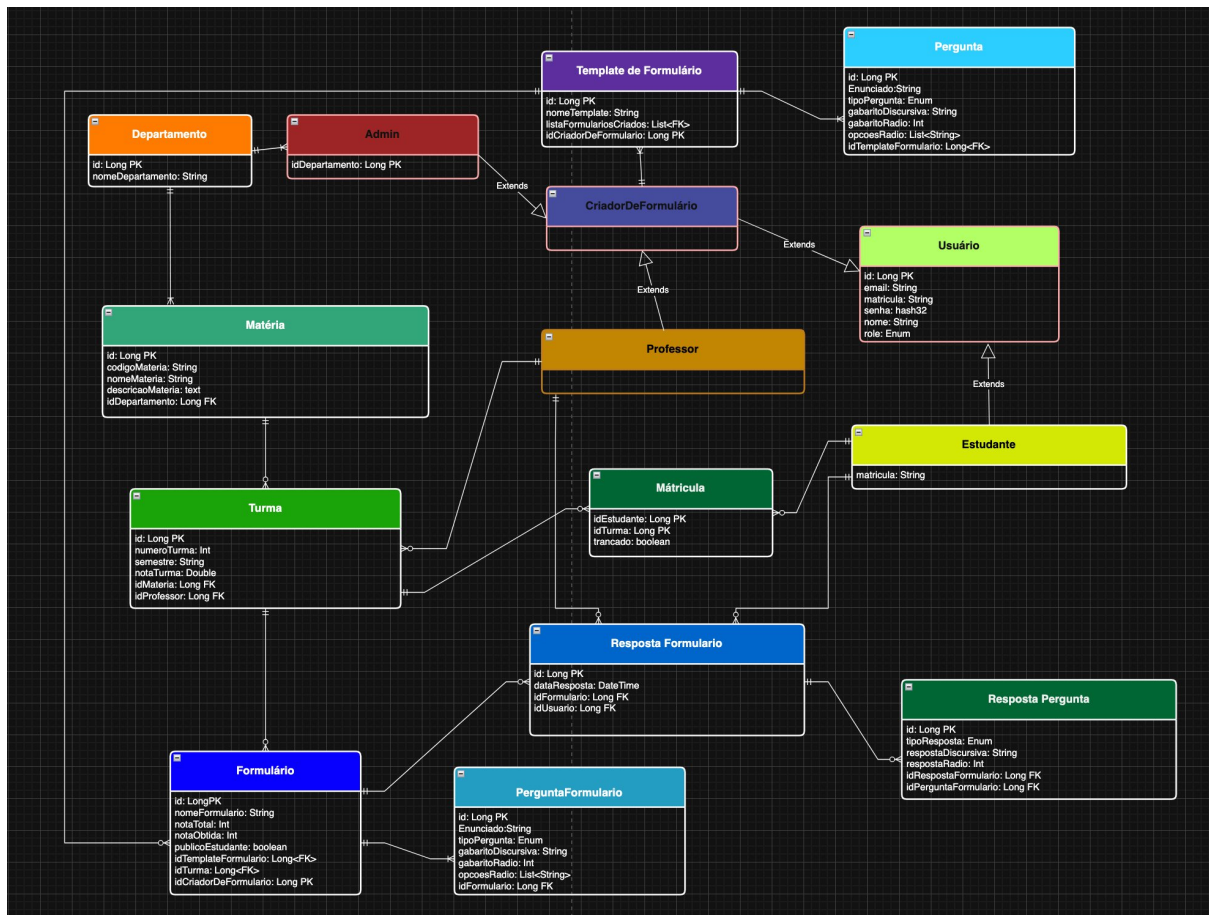


Figura 1: Diagrama Entidade-Relacionamento do sistema CAMAAR.

Diagrama ER

Abaixo está o diagrama entidade-relacionamento que representa a estrutura de dados completa do sistema **CAMAAR**:

Spike Técnico — Implementação das Views em Rails com base no Figma

(Ponto extra)

A investigação técnica teve como objetivo analisar como as telas do protótipo disponibilizado no Figma podem ser implementadas no *Ruby on Rails* durante as próximas etapas do projeto.

No Rails, as telas podem ser desenvolvidas utilizando arquivos `.html.erb`, que permitem combinar HTML com código Ruby. Cada página do sistema é associada a uma *action* de um *controller*, seguindo o padrão MVC do *framework*. Dessa forma, o *controller* fica responsável por buscar e preparar os dados, enquanto a *view* apresenta essas informações ao usuário.

Para aproximar a interface do protótipo do Figma, recomenda-se organizar os elementos visuais em componentes reutilizáveis — como cabeçalho, menu, botões, formulários e *cards* — utilizando *partials* do Rails (ex.: `_navbar.html.erb`, `_form.html.erb`, `_card.html.erb`). Isso evita repetição de código e facilita futuras alterações no *layout*.

Em relação à estilização, recomenda-se o uso de **Bootstrap** ou **Tailwind CSS**. O Bootstrap pode acelerar a criação de *layouts* responsivos, botões, tabelas e formulários; já o Tailwind permite maior controle visual e pode facilitar a reprodução mais fiel do design criado no Figma.

As *views* principais provavelmente envolverão formulários, listagens, telas de cadastro e visualização, além de botões de ação. Para formulários, o Rails oferece o *helper form_with*, que simplifica a criação e integração com *models* e *controllers*. Para listagens, pode-se utilizar tabelas ou *cards*, conforme a proposta visual do Figma.

Conclui-se que a implementação das *views* em Rails é viável utilizando a estrutura padrão do *framework*, com arquivos *.html.erb*, *partials* para reaproveitamento de componentes e uma biblioteca CSS para facilitar a estilização. Esta investigação servirá como base para que, na próxima Sprint, o grupo transforme o protótipo visual em telas funcionais.

Conclusão

Este relatório apresentou o modelo entidade-relacionamento do sistema **CAMAAR**, com a descrição de suas quinze entidades, seus atributos e os relacionamentos entre elas. O modelo cobre os dois épicos principais do sistema — **Épico Usuário** e **Épico Formulário** — e foi elaborado com base nas histórias de usuário definidas para esta primeira sprint.

A hierarquia de herança entre **Usuário**, **CriadorDeFormulário**, **Admin**, **Professor** e **Estudante** garante uma organização clara dos papéis e responsabilidades dentro do sistema. A entidade **Matrícula** viabiliza o relacionamento *many-to-many* entre estudantes e turmas, enquanto a separação entre **Template de Formulário** e **Formulário** permite reaproveitamento e padronização na criação de avaliações.

A investigação técnica sobre as *views* em Rails demonstrou que a abordagem com *.html.erb*, *partials* e bibliotecas CSS é viável e servirá de base para as próximas sprints.